SQL stands for Structured Query Language. It is a language used to communicate with relational databases.

Examples of common SQL commands are:

```
SELECT * FROM student;

INSERT INTO student VALUES (101, 'Rahul', 85);

UPDATE student SET marks = 90 WHERE roll = 101;

DELETE FROM student WHERE roll = 101;
```

SELECT retrieves data. **INSERT** adds data. **UPDATE** changes existing data. **DELETE** removes data.

JDBC allows a Java program to send SQL statements to a database and receive results back.

5. What is an API?

This is one of the most important beginner questions.

API stands for **Application Programming Interface**.

An API is a defined way for one piece of software to use the functionality provided by another piece of software or system.

Think of an API as a set of rules, operations, and interfaces that software exposes for other software to use.

Restaurant analogy: Imagine a restaurant. You do not walk into the kitchen and directly control the cooking equipment. You use the menu to choose what you want and give your order to the waiter. The restaurant provides a controlled interface for requesting services.

In programming, an API similarly provides an agreed way for your program to request functionality.

Important: An API is not necessarily a website, and it is not necessarily a physical piece of software. It is an interface/contract that programmers use.

JDBC is a **Java API** for database connectivity. It defines standard classes and interfaces through which Java programs can work with databases.

6. What is JDBC?

JDBC stands for **Java Database Connectivity**.

JDBC is a Java API that allows Java applications to connect to databases, send SQL statements, retrieve results, and manage database-related operations.

Think of JDBC as the standard bridge between a Java application and a database.

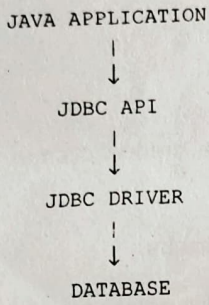
```
Java Application
        |
        | JDBC
        ↓
   JDBC Driver
        |
        ↓
    Database
```

JDBC itself provides the standard programming interface. A database-specific JDBC driver provides the implementation needed to communicate with a particular database.

One-sentence definition: JDBC is the standard Java API used to connect Java applications with databases and perform database operations.

7. JDBC Architecture

A beginner should understand the role of each layer rather than memorizing a diagram.



Java Application: The program written by the developer.

JDBC API: The standard Java interfaces/classes used for database programming.

JDBC Driver: Database-specific software that allows JDBC operations to communicate with a particular database.

Database: The system that stores and processes the data.

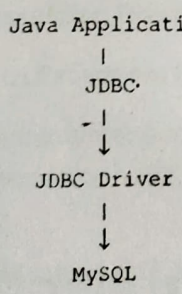
The key idea is: **the Java program uses JDBC; the JDBC driver handles database-specific communication.**

8. What is a JDBC Driver?

A **JDBC driver** is software that enables a Java application using JDBC to communicate with a particular database.

Why is it needed? Because different database systems can have different communication protocols and implementation details.

Think of the driver as a translator between the standard JDBC world and a particular database system.



For MySQL, a commonly used driver is **MySQL Connector/J**. For another database, you use that database's compatible JDBC driver.

Important: A JDBC driver is not the database itself. MySQL is the database system; Connector/J is the JDBC driver used by Java applications to communicate with MySQL.

9. Four Types of JDBC Drivers

Traditional JDBC textbooks describe four driver types.

Type 1 — JDBC-ODBC Bridge Driver: JDBC communicates through ODBC. It is mainly historical now; the JDBC-ODBC Bridge was removed from the JDK in Java 8.

Type 2 — Native-API Driver: Uses the database's native API and native libraries. (Also known as Type-2)

Type 3 — Network Protocol Driver: Communicates with middleware, which then communicates with the database.

Type 4 — Thin/Pure Java Driver: A Java-based driver that communicates directly with the database using its protocol. This is the driver type most relevant to modern JDBC development.

Quick table:

Type	Name	Basic path	
1	JDBC-ODBC Bridge	JDBC → ODBC → Database	sun.jdbc.odbc
2	Native-API	JDBC → Native API → Database	Oracle OCI driver
3	Network Protocol	JDBC → Middleware → Database	
4	Thin / Pure Java	JDBC → Database	Mysql, Oracle, Postgresql

10. JDBC API: Important Classes and Interfaces

JDBC provides many classes and interfaces. Beginners often see these names in programs and wonder what they mean.

DriverManager: Helps manage JDBC drivers and obtain database connections.

Connection: Represents a connection/session between the Java application and database.

Statement: Used to execute SQL statements.

PreparedStatement: Used to execute parameterized SQL statements and is generally preferred when values come from users.

CallableStatement: Used to call stored procedures.

ResultSet: Represents the rows returned by a query such as SELECT.

SQLException: Represents an exception related to SQL/database operations.

DataSource: A standard abstraction for obtaining database connections; it is commonly used with connection pools.

11. What is a Package?

You may see code such as:

```
import java.sql.Connection;
```

The word **package** is a way Java organizes related classes and interfaces.

java.sql is a Java package containing JDBC-related classes and interfaces.

The dot means we are referring to a name inside another name. So **java.sql.Connection** means the Connection type located in the java.sql package.

import tells the Java compiler that we want to use a class/interface by its simple name in our source code.

Without an import, we can also write the fully qualified name, such as **java.sql.Connection**.

12. What are Class, Object, Method, and Interface?

These words appear constantly in Java and JDBC.

Class: A class is a definition/blueprint describing data and behavior

Object: An object is an instance created from a class. It represents a usable entity at runtime.

Method: A method is a named operation/function defined in a class or interface.

Interface: An interface defines a contract of operations that implementing classes provide. JDBC makes extensive use of interfaces.

For example, **Connection** is a JDBC interface. A JDBC driver supplies an implementation that actually performs the database-specific work.

When you see:

```
DriverManager.getConnection(...)
```

the dot is being used to access the **getConnection()** method through **DriverManager**.

13. Establishing a JDBC Connection

Now we can finally understand the connection process without assuming prior knowledge.

A typical sequence is:

1. Make the JDBC driver available to the project (Load & register the JDBC driver)
2. Prepare the database connection information (Define the database URL)
3. Obtain a Connection Create a connection object using DriverManager.getConnection()
4. Create a Statement or PreparedStatement
5. Execute SQL } use the connection for database operations
6. Process the result
7. Close resources

In modern JDBC, compatible drivers are normally discovered automatically when the driver is on the application's classpath. Older textbooks may show an explicit Class.forName(...) step. That step is usually unnecessary with modern JDBC drivers.

Classpath/dependency: The application must have the driver library available to it. In build tools such as Maven or Gradle, this is normally declared as a dependency.

14. Understanding the JDBC URL

A JDBC URL tells JDBC what database system and database location we want to connect to.

For a typical MySQL example:

```
jdbc:mysql://localhost:3306/college
```

→ Port number
→ Database name
→ Host name

Break it into pieces:

jdbc: Indicates a JDBC connection URL.

mysql: Identifies the MySQL JDBC protocol/driver family. (Subprotocol)

localhost: Means the database server is running on the same computer as the application.

3306: The commonly used default MySQL port.

college: The database name.

A **port** is a numbered network endpoint through which a service can accept connections. The exact port can be changed in configuration; 3306 is simply MySQL's common default.

If the database is on another computer, the host part may contain a hostname or IP address instead of localhost.

15. First Complete JDBC Connection Program

Here is a small example whose only purpose is to establish a connection.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DatabaseConnection {
    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/college";
        String username = "root";
        String password = "password";

        try {
            Connection con =
                DriverManager.getConnection(url, username, password);

            System.out.println("Database connected successfully.");

            con.close();

        } catch (SQLException e) {
            System.out.println("Connection failed.");
            e.printStackTrace();
        }
    }
}
```

Important: The username, password, database name, and server details in this example are placeholders. A real application must use the actual configured credentials and should handle secrets securely rather than hard-coding passwords.

16. Reading the Program Line by Line

import java.sql.Connection; — Makes the Connection type available by its short name.

import java.sql.DriverManager; — Makes DriverManager available.

import java.sql.SQLException; — Makes SQLException available.

public class DatabaseConnection -- Defines a class named DatabaseConnection.

public static void main(String[] args) — The standard entry point of a Java application.

String url = ... — Stores the database URL in a String variable.

String username = ... — Stores the database username.

String password = ... — Stores the database password.

try { ... } — Contains code that may produce an exception.

Connection con = ... — Declares a Connection reference and assigns it the connection returned by DriverManager.

DriverManager.getConnection(...) — Requests a database connection using the supplied connection details.

System.out.println(...) — Prints a message to the console.

con.close() — Releases the connection. When using a pool, close generally returns the connection to the pool rather than physically destroying it.

catch (SQLException e) — Handles a SQL-related exception.

e.printStackTrace() — Prints diagnostic information about the exception.

17. Executing SQL and ResultSet

Once a connection exists, we can execute SQL.

```
Statement stmt = con.createStatement();

ResultSet rs =
    stmt.executeQuery("SELECT * FROM student");
```

createStatement() creates a Statement associated with the connection.

executeQuery() is used for queries that return a result set, such as SELECT.

ResultSet represents the returned rows.

To read rows, we commonly use:

```
while (rs.next()) {
    System.out.println(rs.getInt("roll"));
    System.out.println(rs.getString("name"));
    System.out.println(rs.getInt("marks"));
}
```

Think of ResultSet as a cursor over the returned rows. Initially the cursor is before the first row. Each successful **next()** moves to the next row. When there are no more rows, next() returns false.

18. Statement vs PreparedStatement

Statement is suitable for simple SQL that does not need parameters.

```
Statement stmt = con.createStatement();\nResultSet rs = stmt.executeQuery("SELECT * FROM student");
```

PreparedStatement is designed for parameterized SQL.

```
PreparedStatement ps =
    con.prepareStatement(
        "SELECT * FROM student WHERE roll = ?"
    );

ps.setInt(1, 101);
```

```
ResultSet rs = ps.executeQuery();
```

The ? is a placeholder. **setInt(1, 101)** supplies 101 for the first placeholder.

PreparedStatement is generally preferred when values originate from users or other external input because it separates SQL structure from parameter values and helps prevent SQL injection when used correctly.

SQL injection: A security problem in which untrusted input is allowed to alter the intended SQL statement. Parameterized queries are a primary defense.

19. Exceptions and SQLException

Programs do not always succeed. A database may be unavailable, credentials may be wrong, the SQL may be invalid, or a network problem may occur.

Java uses **exceptions** to represent many kinds of abnormal conditions.

SQLException is a Java exception type specifically associated with database access errors and other SQL-related problems.

That is why JDBC code commonly uses try/catch or another appropriate exception-handling mechanism.

```
try {
    Connection con =
        DriverManager.getConnection(url, username, password);
} catch (SQLException e) {
    e.printStackTrace();
}
```

Exception handling does not make errors disappear. It gives the program a controlled way to respond to errors.

20. Closing Resources

Database resources should be released when we are finished using them.

Common resources include Connection, Statement/PreparedStatement, and ResultSet.

Modern Java code often uses **try-with-resources** so that resources are closed automatically.

```
try (Connection con =
        DriverManager.getConnection(url, username, password)) {

    // use the connection here

} catch (SQLException e) {
    e.printStackTrace();
}
```

For multiple resources, each resource can be declared in the try-with-resources header. This is usually safer than relying on manually calling close() on every resource in every execution path.

21. Connection Pooling

Now we reach the final major topic: **connection pooling**.

Opening a database connection is not free. It can involve network communication, authentication, session setup, and database-side work.

If an application creates a brand-new physical connection for every small request and immediately destroys it, a large application can waste resources.

A **connection pool** maintains a collection of reusable database connections.

```
        CONNECTION POOL
+-------------------------+
| C1  Available           |
| C2  In Use              |
| C3  Available           |
| C4  Available           |
| C5  In Use              |
+-------------------------+
```

When the application needs a connection, it obtains an available connection from the pool. When it finishes, it returns the connection to the pool.

```
Application
    |
    | get connection
    v
Connection Pool
    |
    | give available connection
    v
Application uses it
    |
    | close/return
    v
Connection Pool
```

Benefits: *reduced connection-creation overhead, controlled maximum connections, better resource utilization, and improved scalability for applications with many requests.*

22. DataSource and Connection Pooling

In many modern Java applications, a **DataSource** is used as the standard abstraction for obtaining connections.

```
Connection con = dataSource.getConnection();
```

A DataSource implementation may be backed by a connection pool.

```
Application
    |
    ↓
  DataSource
    |
    ↓
Connection Pool
    |
    ↓
 Database
```

The important distinction is that **DataSource is an interface/abstraction for obtaining connections**, while a **connection pool is a resource-management technique**. A particular DataSource implementation may provide pooling.

When a pooled connection is 'closed' by application code, the pool commonly returns it to the pool so it can be reused.

UNIT - III
JDBC (Java Database Connectivity)

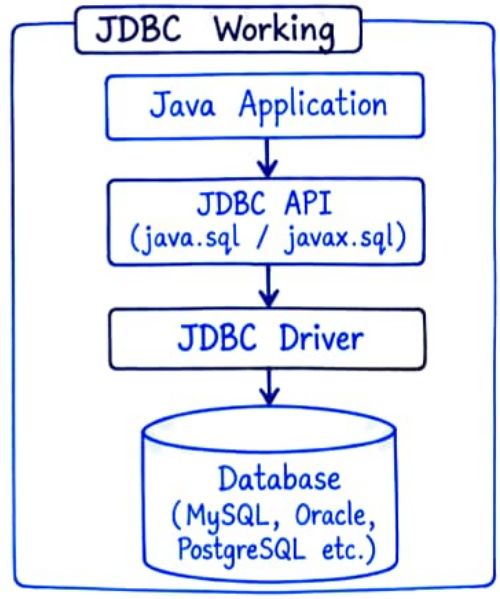
What is JDBC ?

JDBC (Java Database Connectivity) is a standard Java API used to connect Java applications with databases, execute SQL statements and process the results returned by the database.
It provides a common way for Java programs to communicate with different databases such as MySQL, Oracle, PostgreSQL etc.

Why do we need JDBC ?

- To establish a connection between Java application and database.
- To send SQL queries to the database.
- To insert, update or delete data.
- To retrieve data from tables.
- To process the results returned by the database.
- To manage database connections efficiently.

JDBC acts as a bridge between Java Application and Database.



Key Features of JDBC :

- Database Connectivity – Connects Java applications with databases.
- SQL Support – Allows execution of SQL queries and commands.
- Platform Independent – Works on different operating systems.
- Database Independent – Same JDBC API can be used with different databases.
- Result Processing – Provides ResultSet to process retrieved data.
- Transaction Support – Supports commit and rollback operations.
- Exception Handling – Provides database-related exceptions through SQLException.

Important JDBC Packages :

Package	Purpose
java.sql	Contains core JDBC classes and interfaces.
javax.sql	Provides advanced database features such as DataSource and connection pooling.

Simple Example :

```
import java.sql.*;
class Demo {
   public static void main(String[] args)
         throws Exception {
      Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost/test",
            "root", "password");
      System.out.println("Connected!");
      con.close();
   }
}
```

Output :

```
Connected!
```

UNIT - III

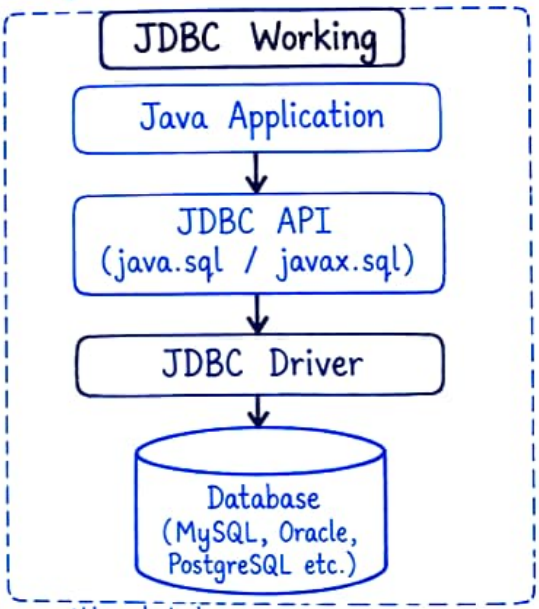
JDBC (Java Database Connectivity)

What is JDBC ?

JDBC (Java Database Connectivity) is a standard Java API used to connect Java applications with databases, execute SQL statements and process the results returned by the database.
It provides a common way for Java programs to communicate with different databases such as MySQL, Oracle, PostgreSQL etc.

Why do we need JDBC ?

- To establish a comnection between Java application and database.
- To send SQL queries to the database.
- To insert, update or delete data.
- To retrieve data from tables.
- To process the results returned by the database.
- To manage database connections efficiently.



Key Features of JDBC :

- Database Connectivity - Connects Java applications with databases.
- SQL Support - Allows execution of SQL queries and commands.
- Platform Independent - Works on different operating systems.
- Database Independent - Same JDBC API can be used with different databases.
- Result Processing - Provides ResultSet to process retrieved data.
- Transaction Support - Supports commit and rollback operations.
- Exception Handling - Provides database-related exceptions through SQLException.

Important JDBC Packages :

Package	Purpose
java.sql	Contains core JDBC classes and interfaces.
javax.sql	Provides advanced database features such as DataSource and connection pooling.

Simple Example :

```
import java.sql.*;
class Demo {
    public static void main(String[] args)
            throws Exception {
        Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost/test",
                "root", "password");
        System.out.println("Connected!");
        con.close();
    }
}
```

Output :

```
Connected!
```

UNIT - III

JDBC (Java Database Connectivity)

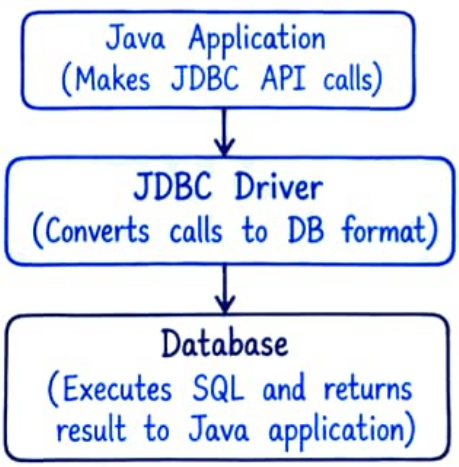
JDBC Drivers

A JDBC Driver is a software component that acts as a bridge between the JDBC API and the database. It converts Java method calls into databasee specific commands and sends them to the database.

Types of JDBC Drivers

Type	Name	Description	Example
Type 1	JDBC-ODBC Bridge Driver	Converts JDBC calls into ODBC calls. Not recommended (Deprecated).	sun.jdbc.odbc.JdbcOdbcDriver (removed in Java 8)
Type 2	Native-API Driver	Converts JDBC calls into database specific native API calls.	Oracle OCI driver
Type 3	Network Protocol Driver	Converts JDBC calls into a middleware specific protocol. Used in 3-tier architecture.	Used with middleware application server
Type 4	Thin Driver	Directly converts JDBC calls into database specific protocol. Pure Java, best performance, widely used.	MySQL, Oracle, PostgreSQL drivers (e.g., com.mysql.cj.jdbc.Driver)

JDBC Driver Working



Steps to Connect to a Database (JDBC Basics)

1. Load and register the Driver (optional in modern versions)
2. Establish the Connection using DriverManager
3. Create a Statement or PreparedStatement
4. Execute SQL query or update
5. Process the ResultSet (for queries)
6. Close the resources (ResultSet, Statement, Connection)

Common JDBC Interfaces and Classes

Interface / Class	Purpose
DriverManager	Provides database connections.
Connection	Represents a connection with database.
Statement	Used to execute simple SQL queries.
PreparedStatement	Used to execute parameterized queries.
CallableStatement	Used to call stored procedures.
ResultSet	Represents the result of a query.
SQLExeeption	Handles database related errors.

Example : Loading and Registering a Driver

```
import java.sql.*;
class LoadDriver {
    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            System.out.println("Driver Loaded Successfully!");
        } catch (ClassNotFoundException e) {
            System.out.println("Driver not found:" + e.getMessage());
        }
    }
}
```

Key Points :

- Class.forName() is used to load and register the JDBC driver.
- In modern JDBC versions (4.0+), explicit registration is not required if the driver is in the classpath.
- For MySQL 8+, use "com.mysql.cj.jdbc.Driver"
- For older versions, use "com.mysql.jdbc.Driver"

UNIT - III

JDBC (Java Database Connectivity)

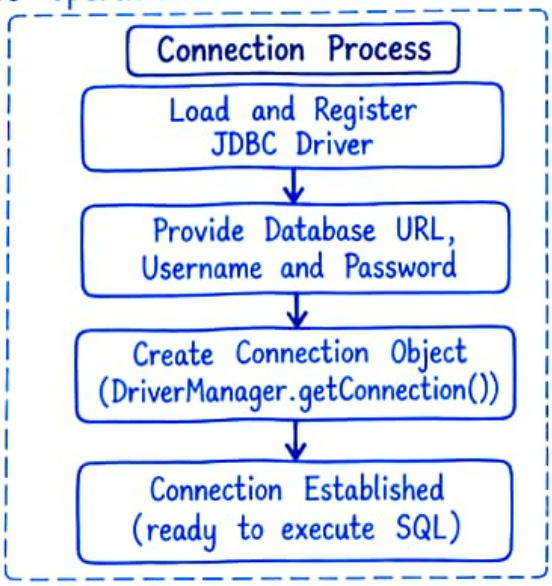
Establishing Connection in JDBC

A connection in JDBC is established using the DriverManager class, which uses a suitable JDBC driver to connect to the database. The connection object is used to send SQL queries and perform database operations.

Steps to Establish Connection

1. Load and register the JDBC driver.
2. Define the database URL.
3. Create a Connection object using DriverManager.getConnection().
4. Use the connection for database operations.
5. Close the connection to release resources.

Connection Process



Database URL Format

```
jdbc:<subprotocol>:<subname>//<host>:<port>/<database>
```

- JDBC identifier
- Subprotocol (e.g. mysql, oracle)
- Database specific name
- Host name (e.g. localhost)
- Port number (e.g. 3306)
- Database name

Examples of Database URL

Database	URL	Example
MySQL	jdbc:mysql://<host>:<port>/<db>	jdbc:mysql://localhost:3306/test
Oracle	jdbc:oracle:thin:@<host>:<port>:<sid>	jdbc:oracle:thin:@localhost:1521:xe
PostgreSQL	jdbc:postgresql://<host>:<port>/<db>	jdbc:postgresql://localhost:5432/test
SQL Server	jdbc:sqlserver://<host>:<port>;databaseName=<db>	jdbc:sqlserver://localhost:1433;databaseName=test

Example : Establishing a Connection (MySQL)

```
import java.sql.*;
class DBConnection {
    public static void main(String[] args) {
        Connection con = null;
        try {
            // 1. Load and register driver
            Class.forName("com.mysql.cj.jdbc.Driver");
            // 2. Define URL, username and password
            String url = "jdbc:mysql://localhost:3306/test";
            String user = "root";
            String password = "password";
            // 3. Create connection
            con = DriverManager.getConnection(url, user, password);
            System.out.println("Connection Established!");
        } catch (ClassNotFoundException e) {
            System.out.println("Driver not found!");
        } catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        } finally {
            try { if(con != null) con.close(); } catch (SQLException e) {}
        }
    }
}
```

Explanation

1. Class.forName() loads and registers the JDBC driver.
2. DriverManager.getConnection() establishes a connection with the database.
3. If successful, a Connection object is returned.
4. Always close the connection in finally block to release resources.

Output :

```
Connection Established!
```

Common Exceptions

Exception	Reason
ClassNotFoundException	Driver class not found.
SQLException	Database connection or SQL error.
NullPointerException	When connection object is null.
IllegalArgumentException	Invalid URL, username or password.

UNIT – III

JDBC (Java Database Connectivity)

Executing SQL Statements in JDBC

JDBC provides three types of Statement objects to execute SQL queries and commands.

Type	Class	Purpose	Important Methods
Statement	java.sql.Statement	Used to execute simple SQL queries. (static queries)	executeQuery(), executeUpdate()
Prepared Statement	java.sql.PreparedStatement	Used to execute parameterized queries. (dynamic queries)	setXXX(), executeQuery(), executeUpdate()
Callable Statement	java.sql.CallableStatement	Used to call stored procedures in the database.	setXXX(), execute(), executeQuery(), executeUpdate()

Execute an INSERT / UPDATE / DELETE Query

```
import java.sql.*;
class UpdateDemo {
    public static void main(String[] args) throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/test", "root", "password");
        Statement stmt = con.createStatement();
        int rows = stmt.executeUpdate("INSERT INTO student VALUES (3,'Amit')");
        System.out.println(rows + " row(s) inserted.");
        stmt.close();
        con.close();
    }
}
```

Output :
1 row(s) inserted.

Using PreparedStatement (Parameterized Query)

```
import java.sql.*;
class PreparedDemo {
    public static void main(String[] args) throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/test", "root", "password");
        String sql = "INSERT INTO student(id, name, age) VALUES (?, ?, ?)";
        PreparedStatement ps = con.prepareStatement(sql);
        ps.setInt(1, 4);
        ps.setString(2, "Neha");
        ps.setInt(3, 20);
        int rows = ps.executeUpdate();
        System.out.println(rows + " row(s) inserted.");
    }   ps.close();
}       con.close();
```

Output :
1 row(s) inserted.

Closing Resources

- Always close ResultSet, Statement/PreparedStatement and Connection after use.
- It releases database and system resources.
- Use try-catch-finally or try-with-resources (Java 7+).

Execute a SELECT Query (using Statement)

```
import java.sql.*;
class SelectDemo {
    public static void main(String[] args)
            throws Exception {
        // 1. Load and register driver
        Class.forName("com.mysql.cj.jdbc.Driver");
        // 2. Establish connection
        Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/test",
                "root", "password");
        // 3. Create statement
        Statement stmt = con.createStatement();
        // 4. Execute query
        ResultSet rs = stmt.executeQuery(
                "SELECT * FROM student");
        // 5. Process ResultSet
        while (rs.next()) {
            int id = rs.getInt(1);
            String name = rs.getString(2);
            System.out.println(id + " " + name);
        }
        // 6. Close resources
        rs.close();
        stmt.close();
        con.close();
    }
}
```

ResultSet Methods (Important)

Method	Purpose
next()	Moves cursor to next row. Returns true if a row exists.
getInt()	Retrieves integer value.
getString()	Retrieves string value.
getDouble()	Retrieves double value.
getBoolean()	Retrieves boolean value.
getDate()	Retrieves date value.
getObject()	Retrieves value as Object.

Using try-with-resources (Java 7+)

```
try (Connection con = DriverManager.getConnection(url, user, pass);
     Statement stmt = con.createStatement();
     ResultSet rs = stmt.executeQuery("SELECT * FROM student")) {
    while (rs.next()) {
        System.out.println(rs.getInt(1) + " " + rs.getString(2));
    }
} catch (SQLException e) {
    e.printStackTrace();
}
```